

Interactive Social Network Analysis

Visualizing Research Interest Connections — UTA School of Social Work

Project Write-Up & Technical Documentation

Project Type:	Data Visualization / Social Network Analysis (SNA)
Built With:	Python (pandas), D3.js v7, HTML5/CSS3, SVG
Data Source:	peopleToInterests_matrixAndEdgelist.xlsx
Deployment:	GitHub Pages (static, client-side rendering)
Network Size:	68 nodes (28 people + 40 interests), 309 connections

1. Project Overview

This project is an interactive Social Network Analysis (SNA) visualization that maps the research interests of faculty, PhD students, postdoctoral fellows, and alumni at the UT Arlington School of Social Work (plus a few affiliated members from partner institutions). The goal was to turn a static spreadsheet of people-to-interest relationships into an explorable, browser-based network graph that reveals who shares research interests with whom, and how interconnected the department's research community really is.

The final deliverable is a single, self-contained HTML file that runs entirely in the browser — no server, database, or installation required — making it easy to host for free on GitHub Pages and share with anyone via a link.

Problem Statement

Spreadsheets are good at storing relational data but terrible at revealing patterns within it. With 28 people and 40 distinct research interests, it is nearly impossible to answer questions like "who else studies trauma and substance use?" or "which faculty member has the most diverse research network?" just by scanning rows and columns. A network graph turns these relationships into something visual and intuitive — clusters, hubs, and outliers become immediately visible.

2. Dataset Description

The source file **peopleToInterests_matrixAndEdgelist.xlsx** contained three sheets:

- **Matrix** — a 28×40 binary adjacency matrix (1 = person has that research interest, 0 = does not).
- **Edgelist** — the same relationships flattened into 309 individual (Person → Interest) pairs, used as the primary data source for graph construction.
- **Member_Info** — biographical details for each of the 28 people: Role, Institution, Profile URL, and Email.

Data was loaded and reshaped using **pandas**, then converted into a JSON structure with three parts:

- **nodes[]** — every unique person and interest, tagged by type (person / interest) and degree (number of connections).
- **edges[]** — every person-to-interest relationship, used to draw the connecting lines.
- **member_info{}** — a lookup table keyed by name, used to populate the click-to-view detail panel for people.

3. Methodology & Technical Approach

3.1 Why D3.js Instead of Pyvis

Pyvis was the original tool considered for this task, since it is a popular Python wrapper for building network graphs. However, the sandboxed build environment had no internet access, so the **pyvis** package could not be installed via pip. Rather than treat this as a blocker, it was used as an opportunity to build the graph directly in **D3.js** — a JavaScript library that Pyvis itself is built on top of internally (Pyvis

wraps vis.js, a sibling library in the same ecosystem).

Building directly in D3.js turned out to unlock capabilities that Pyvis does not expose:

- **Custom interactive side panel** showing role, institution, email, and connection list on click — not possible with Pyvis's basic hover tooltips.
- **Clickable connection chips** that let users navigate the network node-to-node without losing context.
- **Live search bar** with autocomplete that pans and zooms directly to a selected node.
- **Fine-grained physics control** (repulsion strength, link distance, collision radius) to prevent node overlap — Pyvis only exposes a small subset of these parameters.
- **Full custom styling** — rounded "pill" nodes for people vs. square nodes for interests, with size scaled dynamically by connection count.

Since D3.js loads directly from a CDN inside the HTML file, no installation or build step is required — the output is just as portable as a Pyvis export, but with significantly more functionality.

3.2 Force-Directed Layout

The graph uses D3's **force simulation** engine, which treats every node like a physical particle:

- **Link force** — pulls connected nodes toward each other, with distance scaled by combined connection count so densely-connected nodes spread out naturally.
- **Charge force (repulsion)** — pushes all nodes away from each other (strength: -900) to prevent crowding.
- **Collision force** — enforces a minimum buffer zone (node radius + 32px) around every node so labels and shapes never overlap.
- **Centering force** — keeps the overall cluster centered in the viewport.

These four forces run simultaneously every animation frame until the layout reaches a stable equilibrium — this is what produces the organic, cluster-like arrangement where closely related people and interests naturally group together.

3.3 Visual Encoding

Visual Property	Encodes	Example
Node shape	Type of entity	Circle = Person · Square = Interest
Node color	Type of entity	Orange = Person · Blue = Interest
Node size	Number of connections (degree)	Larger node = more connections
Line thickness/opacity	Relationship strength on hover	Highlighted vs. dimmed on selection

4. Key Features

- **Click-to-inspect panel** (bottom-right) — shows full details for any node: type, total connections, and a clickable list of every connected node.
- **Person-specific details** — clicking a person additionally reveals Role, Institution, Email (mailto link), and Profile URL pulled from the Member_Info sheet.
- **Neighbor highlighting** — selecting a node highlights it and all directly connected nodes while dimming the rest of the network.
- **Live search** — type any name or interest to instantly locate and zoom to that node.
- **Pan & zoom** — scroll to zoom, drag canvas to pan, drag individual nodes to reposition them.
- **Reset View / Fit to Screen / Pause Motion** controls in the top toolbar for easier navigation.
- **Fully responsive** — adapts to different screen sizes and window dimensions.

5. Technical Architecture

- **Data layer:** Python + pandas reads the Excel workbook and transforms it into a JSON object (nodes, edges, member_info) embedded directly in the HTML.
- **Rendering layer:** D3.js v7 (force simulation + SVG rendering) draws and animates the graph.
- **Interaction layer:** Vanilla JavaScript event listeners handle click, hover, drag, search, and zoom interactions.
- **Presentation layer:** Custom CSS for the toolbar, legend, tooltip, and floating info panel.

Because everything — data, logic, and styling — lives in one HTML file with D3.js loaded from a public CDN, the project has zero backend dependencies and can be hosted on any static file host (GitHub Pages, Netlify, Vercel, or even opened locally as a file).

6. Deployment

The graph is deployed as a static site using **GitHub Pages**:

- The HTML file is renamed to **index.html** and pushed to a GitHub repository.
- GitHub Pages is enabled under Settings → Pages → Deploy from branch (main / root).
- The site becomes publicly accessible at **https://<username>.github.io** within a few minutes, with no server costs or maintenance.

7. Skills Demonstrated

- **Data wrangling** — cleaning and reshaping multi-sheet Excel data with pandas into a graph-ready JSON structure.
- **Data visualization** — applying force-directed graph theory and visual encoding principles (size, color, shape) to communicate network structure.
- **Front-end development** — building an interactive UI (search, click-to-inspect panel, tooltips) with vanilla JavaScript and D3.js.

- **Problem-solving under constraints** — adapting the technical approach (Pyvis → D3.js) when the original plan hit an environment limitation.
- **Deployment & hosting** — shipping a working, publicly accessible product via GitHub Pages.

8. Possible Future Improvements

- Add filtering by Role (Professor, PhD Student, Postdoc, etc.) or Institution.
- Add a community-detection algorithm to auto-color clusters of closely related researchers.
- Add an export feature to download the current view as a PNG/SVG image.
- Add a "shortest path" finder to show how any two people are connected through shared interests.
- Migrate to a lightweight backend so the underlying dataset can be updated without editing the HTML file directly.

Prepared as project documentation for the Interactive Social Network Analysis visualization.